Durante el desarrollo de los servicios de contactos y tareas, mi enfoque de pruebas unitarias se adaptó directamente a los requisitos establecidos para cada servicio. En el caso del servicio de contactos, las pruebas se enfocaron en las operaciones principales que debía proporcionar el sistema, como agregar, eliminar y actualizar información de un contacto. También consideré condiciones que podían producir errores, como intentar agregar un contacto con un identificador que ya existía o modificar un identificador que debía permanecer sin cambios. Por ejemplo, la prueba testCannotAddDuplicateContactId verifica que el servicio rechace contactos duplicados mediante assertThrows(IllegalArgumentException.class, () -> ...). Este tipo de prueba permite comprobar tanto el comportamiento esperado como las condiciones de error. Según García (2017), las pruebas unitarias permiten verificar que las unidades individuales de software cumplan con los comportamientos definidos en los requisitos.

La calidad general de mis pruebas JUnit se puede observar en la variedad de casos incluidos y en los resultados obtenidos al ejecutarlas. En ContactServiceTest incluí pruebas para agregar contactos, evitar identificadores duplicados, eliminar contactos y actualizar diferentes atributos, incluyendo nombre, apellido, teléfono y dirección. También incorporé una prueba para impedir la modificación del identificador del contacto. Al ejecutar el proyecto con Maven, las pruebas finalizaron correctamente, mostrando cero fallos y cero errores. Sin embargo, la ejecución correcta de las pruebas no significa necesariamente que todo el código haya sido probado. La cobertura de código es una métrica útil para conocer qué partes del programa fueron ejecutadas durante las pruebas, aunque una cobertura elevada por sí sola no garantiza que las pruebas sean completas o de buena calidad (Myers et al., 2011). Por esta razón, el porcentaje exacto de cobertura debe comprobarse mediante una herramienta como JaCoCo antes de presentar un valor específico.

Para asegurar que mi código fuera técnicamente sólido, utilicé pruebas independientes para las diferentes operaciones del servicio y empleé assertions apropiadas para cada situación. Por ejemplo, en una de las pruebas primero se agrega el contacto mediante service.addContact(contact); y posteriormente se actualiza su teléfono con service.updatePhone("12345", "3019876543");. Después, una assertion permite comprobar que el resultado coincide con el valor esperado. También utilicé assertEquals para comparar resultados y assertThrows para comprobar que las operaciones inválidas generaran la excepción correspondiente. JUnit proporciona estas herramientas para expresar de manera clara las condiciones que deben cumplirse durante una prueba (JUnit Team, 2026). La separación de las pruebas en métodos como testAddContact, testDeleteContact, testUpdateFirstName, testUpdateLastName, testUpdatePhone y testUpdateAddress también facilitó la identificación de errores específicos.

Finalmente, procuré que el código de las pruebas fuera eficiente evitando repetir innecesariamente la configuración inicial. Para ello utilicé el método @BeforeEach setUp(), que prepara el objeto ContactService antes de cada prueba. De esta manera, cada prueba comienza con un estado conocido e independiente, sin tener que repetir la misma configuración en todos los métodos. La utilización de métodos de preparación y assertions específicas ayuda a mantener las pruebas organizadas y facilita su mantenimiento a medida que el software evoluciona. Además, las pruebas utilizan estructuras sencillas y operaciones directamente relacionadas con el requisito que se está verificando. Según Pressman y Maxim (2020), las pruebas deben diseñarse de manera sistemática para detectar errores y proporcionar información sobre la calidad del software.

La experiencia con JUnit me permitió comprender que una buena prueba unitaria no consiste solamente en comprobar que el código funciona, sino también en verificar diferentes escenarios y detectar comportamientos incorrectos antes de integrar el software. La combinación de casos positivos, casos negativos, assertions y pruebas de excepciones proporcionó una forma estructurada de validar los servicios de contactos y tareas. Esta experiencia también me ayudó a comprender la importancia de relacionar las pruebas con los requisitos del sistema y de utilizar métricas como la cobertura como apoyo para evaluar qué tan completo es el conjunto de pruebas.

Referencias

García, B. (2017). *Mastering software testing with JUnit 5*. Packt Publishing.

JUnit Team. (2026). *JUnit 5 user guide*. JUnit. https://docs.junit.org/current/user-guide/

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). John Wiley & Sons.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.